PollenFX - Particle Engine V1.0

Documentation

1. Introduction

PollenFX is a lightweight, clientside, code-first, javascript based, DOM-oriented particle engine. It is used to create a wide variety of visually compelling particle effects within the browser. The tool consists of many different configuration components that allow the developer to dynamically assemble custom particle effects. The engine does not rely on any external css, javascript or JQuery. In order to use PollenFX, one is expected to have minimal (vanilla) javascript knowledge, although the components are fairly simple and well documented along with code snippets and examples. The order of the listed properties of the common objects of PollenFX are also the the order in which they are expected to be provided to the constructor.

- Code-first engine
- Lightweight
- One single .js file
- No CSS files included
- No external scripts/dependencies
- Modular emitter creation
- Well documented with code examples
- Flexible for programatical extensions

---

2. A simple code snippet & general overview

In the first image we like to offer you an easy template to get started with the engine. Firstly, we subscribe to the DOMContentLoaded event with a callback. This means that our provided callback will be executed once the DOM is built. Within this callback we call the start function. The start function resembles a single iteration of our particle effect core loop. It requests one parameter which resembles the milliseconds that have passed since the application started. We call this 'nowTime' in this example. This function is called with a 0 provided since 0 milliseconds have passed when the DOM is only just built. Within the start function we are going to make use of the window.requestAnimationFrame() function, which is the browser's built-in loop function, and is going to continuously be called by the browser depending on how fast your CPU functions. To this requestAnimationFrame we are going to subscribe once again with the start function but in this case the nowTime will be provided by the browser and is going to be the updated passed time in milliseconds. What this essentially means is that we have created an endless loop of the start() function. It this within this function we are going to have our particle emitters do their work.

```
/* Start upon DOM loaded */
document.addEventListener('DOMContentLoaded', () => {
    this.start(0);
});

/* Start the core logic loop */
function start(nowTime) {
    window.requestAnimationFrame(this.start);
}
```

Next, we quickly gloss over the different concepts, after which they will be explained in more depth. First an FXManager is created, which serves as an object that bundles different emitters. Next, we retrieve the element we want to attach our particle effect to. Then we create an origin object in order to define the area in which particles can spawn. An emitter is then created, with this origin injected. Data objects & behavior objects are being created in order to configure the emitter, and then applied to this emitter. The emitter is then inserted into the FXManager, we start the looping function and subscribe to this function by having our fxmanager perform the act() function. This is the general flow of how to use the engine and one will mostly add/alter more data and behavior objects. Bear in mind that in the image below not all constructor parameters are provided and there is alot more to cover. In the next segments these concepts will be explained in more depth, and the full extend of those parameters will be explained.

```
let fxManager;

/* 1. Setup */
document.addEventListener('DOMContentLoaded', () => {
    fxManager = new FXManager(false);

    const anchor = document.getElementById("div1");
    const origin = new CircularEmitterOrigin(50, 0, 100, 40, -1, -1, true, anchor);
    const emitter = new EmitterShoot(origin, 100, 400, 2000, 500, 2)

    /* Data objects */
    const defaultData = new ParticleDefaultData(30, 30, origin, 2, 2, true);
    const cssData = new ParticleCustomCssData("background-color:#eb4034; border:1px solid black; border-radius:100px;", 30, 500);
    const directionData = new ParticleDirectionData(90, 10, 45, 2);
    const rotationData = new ParticleRotationData(45, 20);
    const colorData = new ParticleColorfilterData(210, new Color('#9a11d9'), 100, -1, 2, -1)
    emitter.addParticleData(defaultData);
    emitter.addParticleData(cssData);
    emitter.addParticleData(directionData);
    emitter.addParticleData(rotationData);
    emitter.addParticleData(colorData);

    /* Behavior [objects */
    const sizeByLifeBehavior = new ParticleSizeByLifeBehavior([0, 1, 0], [0, 1, 0]);
    const directionBehavior = new ParticleDirectionalBehavior();
    const gravityBehavior = new ParticleGravityBehavior(10, 3);
    emitter.addParticleBehavior(sizeByLifeBehavior);
    emitter.addParticleBehavior(directionBehavior);
    emitter.addParticleBehavior(gravityBehavior);

    fxManager.addEmitter(emitter, "my_emitter");
    this.start(0);
});

/* Core loop */
function start(nowTime) {
    fxManager.act(nowTime);
    window.requestAnimationFrame(this.start);
}
```

1. FXManager and Emitters

Best explained together are the FXManager and Emitters. An emitter is considered the manner in which particles of the same type are being excreted. It is important to note that an emitter will always create particles of the same type and with the same configuration. An example of this would be the way in which smoke particles are generated one after the other. One could consider this to be a smoke 'emitter'. Particle effects will in many cases be more complex than a singe emitter. A good analalogy would be a campfire. When thinking about a campfire one could imagine the sparks, the smoke and the flames. These three concepts will obviously behave differently, thus requiring three different emitters and this is where the FXManager comes into play. The FXManager is a tool to bundle different emitters in order to create one coherent particle effect. In the analogy described previously an FXManager could combine a smoke, spark and flame emitter, creating a fire particle effect. The creation and different types of emitters will be explained further down.

2. Origins

Regardless of the type of emitter one picks, it is required to also configure an origin object. An origin can be seen as an imaginary or invisible region of space in which particles are

allowed to spawn. Upon creation of an emitter, one is required to pass the origin object to the emitter for it to function correctly.

3. Behavior objects
Behavior objects are being created and provided to the emitters. These behavior objects define the way an individual particles of a certain emitter should behave, and once applied to an emitter, this behavior will be taking effect on all of the particles created from this emitter. As an example, lets consider a blizzard. When snow falls it is obviously affected by gravity and in order to model this, we will add a gravitybehavior object. Secondly, the falling snow is also affected by wind, which can be achieved by adding windbehavior on top. Finally, in case we want the snow to melt whilest falling we could add size-by-life behavior which will change the size of the particle over a certain period of time. It is important to note that behavior objects function independantly of eachother and the endresult is a summation of these individual behaviors combined.

4. Data objects
Going hand in hand with the behavior objects, are the data objects. Data objects contain the data required for a particle to be shown in a certain way. In the same way that behavior objects must be provided to an emitter, one must do so too for data objects. When applying behavior to an emitter it is actually this data object that is internally being altered. It is however important to note that you can apply behavior without its respective data object, in which case a default data object will be created for this behavior type. Applying data objects without their respective behavior objects is also possible and will result in giving the particle an initial, and unchanging property or state. Finally and most intrestingly, data objects and their respective behavior objects can be used in combination and result in an initial configured state for this particle, which is then being altered over time by the behavior object. It is important to note that when talking about 'its respective behavior'-object, that each behavior object acts on a certain data object. An example of this would be the directionbehavior acting on the directiondata. There are also certain data objects that are not acted upon by any behavioral objects.

In the next chaptors we discuss the individual components of PollenFX. Some of them occur in the image shown above, but for those that do not, the way they are being used is very similar. It is important to note that the order in which the properties of those components are listed, is the same order in which you are required to provide them to the constructors upon creating those components. At the end of this document you will find a more extensive code example in which more of these components will be used.

---

3. FXManager

As explained earlier, the FXManager is the core object of PollenFX. One will usually only need one to be active and a one time.

*** allowDOMOverflow (optional):** Defines whether particles will be hidden upon overflowing the body, or whether scrolling will be enabled. If for some reason one would need two FXManager objects, the last FXManager object created will define which allowDOMOverflow value will take action. Default = false which indicates that particles overflowing the body will not be visible.

*** debug (optional):** When set to true, this will set FXManager.devConfig.DEBUG to true, in a similar manner as one would statically set the variable. Beware that for multiple FXManagers, the field will be set to the value that the latest created FXManager picked, which was not null. Default = null and means that no true nor false will be set. It will be ignored.

---

4. Origins

An origin is considered an area in which particles can spawn and have to be passed on to a future emitter in order to function. There are four types of origins.
It is important to note that an origin object can be provided with an optional anchor element. This is a html element to which the emitter and origin become attached. When no anchor is provided, this becomes the body, so make sure the DOM contains one. When the

emitter recieves its origin object, this anchor element will be copied and set to mimic this anchor element. We refer to this copy as the emitterbox. This emitterbox can be interpreted as the container in which the origin objects reside.

1. CircularEmitterOrigin 
Allows particles to come to life somewhere within an elliptical/circular area.

*** posX:** Numeric value representing the center x coordinate of the circular/elliptical area.
*** posY:** Numeric value representing the center y coordinate of the circular/elliptical area.
*** width:** Numeric value representing the width of the circular/elliptical area.
*** height:** Numeric value representing the height of the circular/elliptical area.
*** posXNoise (optional):** Numeric value adding noise/random offset to the posX parameter. Smaller values yield better results. Default = -1 and indicates no noise.
*** posYNoise (optional):** Numeric value adding noise/random offset to the posY parameter. Smaller values yield better results. Default = -1 and indicates no noise.

2. LineEmitterOrigin 
Allows particles to come to life at some point on, or around a defined line segment.

*** posX_1:** Numeric value representing the x position coordinate of first point defining the line segment.
*** posY_1:** Numeric value representing the y position coordinate of first point defining the line segment.
*** posX_2:** Numeric value representing the x position coordinate of second point defining the line segment.
*** posY_2:** Numeric value representing the y position coordinate of second point defining the line segment.
*** posXNoise (optional):** Numeric value which adds noise/random offset to the posX_1 & posX_2 parameters. Smaller values yield better results. Default = -1 and indicates no noise.
*** posYNoise (optional):** Numeric value which adds noise/random offset to the posY_1 & posY_2 parameters. Smaller values yield better results. Default = -1 and indicates no noise.
*** offset (optional):** Numeric value which allows particles to spawn some distance 'offset' away from the line segment, perpendicularly. Default = -1 indicates no offset.

3. PointEmitterOrigin
Allows particles to come to life on a point.

*** posX:** Numeric value representing the x position coordinate of the point defining where particles can spawn.
*** posY:** Numeric value representing the y position coordinate of the point defining where particles can spawn.
*** posXNoise (optional):** Adds noise/random offset to the posX parameter. Smaller values yield better results. Default = -1 and indicates no noise.
*** posYNoise (optional):** Adds noise/random offset to the posY parameter. Smaller values yield better results. Default = -1 and indicates no noise.

4. RectangularEmitterOrigin 
Allows particles to come to life somewhere within a rectangular area.

*** posX:** Numeric value representing the x position coordinate of the top left corner of the rectangular/square in which the particles can spawn.
*** posY:** Numeric value representing the y position coordinate of the top left corner of the rectangular/square in which the particles can spawn.
*** width:** Numeric value representing width of the rectangular/square area.
*** height:** Numeric value representing height of the rectangular/square area.
*** posXNoise (optional):** Numeric value which adds noise/random offset to the posX parameter. Smaller values yield better results. Default = -1 means indicates no noise.
*** posYNoise (optional):** Numeric value which adds noise/random offset to the posY parameter. Smaller values yield better results. Default = -1 means indicates no noise.

Additional origin parameters
Regardless of the chosen origin type, all origins share these parameter options and are

provided after the parameters as described above. These parameters will rarely be modified, but allows for more precise alterations. The order remains respected.

*** overflow (optional):** Boolean value defining whether the particles will be shown when they go beyond the bounds of its emitterbox. This is usually irrelevant for origins without anchorElement. Default = true and indicates visible overflow.
*** anchorElement (optional):** A native HTMLElement object to which the origin is going to be attached. If initialized, the emitterbox will mimic this anchor element. Default = the body-element.
*** containerUnitWidth (optional):** A positionUnit enum value representing whether the 'width' value of this emitterbox should be expressed as a percentage of the anchorElement's width, absolutely in pixels, or as a css vw value. Read further for PositionUnit options. Default = PositionUnit.PERCENTAGE.
*** containerUnitHeight (optional):** A positionUnit enum value representing whether the 'height' value of this emitterbox should be expressed as a percentage of the anchorElement's height, absolutely in pixels, or as a css vh value. Read further for PositionUnit options. Default = PositionUnit.PERCENTAGE.
*** containerWidth (optional):** Numeric value representing the width of the emitterbox, mimicing the anchorElement. The width value is expressed through the containerUnitWidth field. Default = 100.
*** containerHeight (optional):** Numeric value representing the height of the emitterbox, mimicing the anchorElement. The height value is expressed through the containerUnitHeight field. Default = 100.
*** positionUnitWidth (optional):** A positionUnit enum value representing whether the 'left' value of this emitterbox should be expressed as a percentage of the anchorElement's width, absolutely in pixels, or as a css vw value. Default = PositionUnit.PIXEL.
*** positionUnitHeight (optional):** A positionUnit enum value representing whether the 'top' value of this emitterbox should be expressed as a percentage of the anchorElement's height, absolutely in pixels, or as a css vw value. Default = PositionUnit.PIXEL.
*** left (optional):** Optional numeric value representing x-offset added to the emitterbox. The left value is expressed through the positionUnitWidth field. Default = 0.
*** top (optional):** Optional numeric value representing y-offset added to the emitterbox. The top value is expressed through the positionUnitHeight field. Default = 0.
*** includeMargin (optional):** Boolean value defining whether the emitterbox is also going to get the same margin as the anchor element applied or not. Procentual width,height,top and left will be calculated with margin in/excluded depending on this value. Default is false.
*** includePadding (optional):** Boolean value defining whether the emitterbox is also going to get the same padding as the anchor element applied or not. Procentual width,height,top and left will be calculated with padding in/excluded depending on this value. Default is true.
*** includeBorder (optional):** Boolean value defining whether the emitterbox is also going to get the same border as the anchor element applied or not. Procentual width,height,top and left will be calculated with border in/excluded depending on this value. Default is true.

---

5. Emitters

Emitters are considered the source from which particles are being 'emitted' and define the manner in which these particles are being excreted. There are two types of emitter; the burst-emitter and the shoot-emitter. In this section we will go over the different parameters one can configure.

1. EmitterShoot
The EmitterShoot object is an emitter that emits particles one by one in a linear flow. This can last indefinitely or be halted at some point. An example would be a dripping fosset.

*** emitterOrigin:** The origin object that defines the area in which particles are allowed to be created.
*** particleCount:** Numeric value representing the amount of particles that the emitter will emit over its duration. Irrelevant for a duration of -1.

*** emitterDuration:** The time (in ms) over which the configured amount of particles will be spawned. A value of -1 makes the emitter emit particles indefinitely. In this case, the spawnIntervalTime parameter becomes mandatory.
*** particleLifetime:** Defines how long (in ms) particles will remain alive for.
*** delay (optional):** Defines an optional delay (in ms) before the emitter will start. Default = -1 and indicates no delay.
*** particleLifetimeNoise (optional):** Numeric value which adds noise to the particleLifetime of each individual particle. Lower values yield better results. Default = -1 and indicates no noise.
*** spawnIntervalTime (optional):** Only relevant for emitterDuration values of -1, and define the time (in ms) between each particle spawn.

2. EmitterBurst 
The EmitterBurst object is an emitter that releases all of its particle in one go. These bursts can go on indefinitely or occur only once. An example would be a firework explosion.

*** emitterOrigin:** The origin object that defines the area in which particles are allowed to be created.
*** particleCount:** Numeric value representing the amount of particles that the emitter will burst.
*** burstCount:** Numeric value representing the amount of burst events that will occur, each containing as many particles as defined in particleCount.
*** emitterDuration:** Defines the time (in ms) over which the emitter will emit its x burst-events.
*** particleLifetime:** Defines how long (in ms) particles will remain alive for.
*** delay (optional):** Defines an optional delay (in ms) before the emitter will start. Default = -1 and indicates no delay.
*** particleLifetimeNoise (optional):** Adds noise to the particleLifetime of each individual particle. Lower values yield better results. Default = -1 and indicates no noise.
*** burstIntervalTime (optional):** Only relevant for emitterDuration values of -1, and define the time (in ms) between each burst event.

---

6. Data objects

A data object defines a certain property of each particle within an emitter. A data object without respective behavior object represents its fixed state for that property. Adding a behavior object of that respective data object, means that the data object defines its initial state for that property, to then be altered over time by the behavior object.

1. ParticleDefaultData 
In order for a particle to just be shown, it requires at least two properties; position and size. This is what the ParticleDefaultData object prescribes.

*** width:** Numeric value defining the width (in pixels) of the particle.
*** height:** Numeric value defining the height (in pixels) of the particle.
*** emitterOrigin:** The origin object that defines the area in which particles are allowed to be created.
*** noiseWidth (optional):** Numeric value which adds/subtracts random noise to the width property in order to apply randomness to its width. Lower values yield better results. Default = -1 and indicates no noise.
*** noiseHeight (optional):** Numeric value which adds/subtracts random noise to the height property in order to apply randomness to its height. Lower values yield better results. Default = -1 and indicates no noise.
*** uniformSizeWidth (optional):** Boolean value defining whether the optionally added size noise, if provided, should always keep the sizing uniform. If true, the noiseWidth value will be taken as a reference. Default is false, indicating that a non-uniform sizing is allowed.
*** horizontalPivot (optional):** Pivot enum value defining where the pivot point lies, horizontally, in defining the origin of the particle. Default = Pivot.CENTER.
*** verticalPivot (optional):** Pivot enum value defining where the pivot point lies, vertically, in defining the origin of the particle. Default = Pivot.CENTER.

2. ParticleCustomCSSData
The ParticleCustomCSSData allows for custom css stylerules to be added to each particle within an emitter.

*** customCssObject:** String value containing n css stylerules, applied to all particles within the emitter. Beware that each stylerule within this string should end width a semicolon.
*** minZIndex (optional):** A numeric value. Along with the maxZIndex, a random z-index will be assigned between those two values. Default is -1, meaning unset and results in a fixed z-index of 2001.
*** maxZIndex (optional):** A numeric value. Along with the minZIndex, a random z-index will be assigned between those two values. Default is -1, meaning unset and results in a fixed z-index of 2001.

3. ParticleDirectionData
The ParticleDirectionData defines the direction and speed of the particles of an emitter. Because this is a data object, this object by itself won't do anything. It is however altered by many Behavioral objects and serves as an initial state of direction and speed when being altered by these behavioral objects.

*** directionAngle:** Numeric value which defines (in degrees) the direction particles should move toward.
*** speed:** Numeric value which defines the speed by which the particles move in the direction of the provided directionAngle parameter.
*** coneNoise (optional):** Numeric value which adds an imaginary cone (in degrees) around the directionAngle overriding the directionAngle field by a random angle within this cone. Default = -1 and indicates no coneNoise.
*** speedNoise (optional):** Numeric value which adds noise/random offset to the speed parameter. Smaller values yield better results. Default = -1 and indicates no speedNoise.

4. ParticleRotationData
The ParticleRotationData object defines the rotation property of the particles spawned from an emitter.

*** rotation:** Numeric value which defines (in degrees) the amount of rotation a particle must have.
*** coneNoise (Optional):** Numeric value which adds an imaginary cone (in degrees) around the rotation angle and randomizes the rotation value by a random angle within this cone. Default = -1 and means no randomization.

5. ParticleOpacityData
The ParticleOpacityData object defines the transparency of the particle object.

*** opacity:** Numeric value defining the transparency/opacity of the particles of the emitter. Values are meaningful between [0,1].
*** opacityNoise (Optional):** Numeric value which adds noise/random offset to the opacity parameter. Smaller values yield better results. Default = -1 and indicates no opacityNoise.

6. ParticleImageData
The ParticleImageData object allows for particles to be presented as an image.

*** url:** String value representing the URL to the image.
*** imgWidth (optional):** Numeric value representing the physical width of the image file in px. Providing this value might increase performance. Default = -1 and indicates auto-calculation.
*** imgHeight (optional):** Numeric value representing the physical height of the image file in px. Providing this value might increase performance. Default = -1 and indicates auto-calculation.
*** imageFitting (Should not be altered):** The ImageFitting enum value defines how the image is applied to the particle div element and should usually not be altered. Default = ImageFitting.FIT.
*** keyOverride (Should not be altered):** Should not be altered. Flipbookdata extends

Imagedata and overrides this key.
*** imgLoadingCallback (Should not be altered):** Should not be altered. Represents the callback method processing the image.

7. ParticleFlipbookData 
The ParticleFlipbookData object allows for spritesheets/flipbooks to be applied to the particle through which is being flipped in order to display an animation. This data object should be used along with ParticleFlipbookBehavior, in order to to achieve a meaningful animation.

*** url:** String value representing the URL to the image.
*** imgWidth (optional):** Numeric value representing the physical width of the image file in px. Providing this value might increase performance. Default = -1 and indicates auto-calculation.
*** imgHeight (optional):** Numeric value representing the physical height of the image file in px. Providing this value might increase performance. Default = -1 and indicates auto-calculation.
*** frameCountX:** Numeric value representing the amount of images within the provided spritesheet over the x axis.
*** frameCountY:** Numeric value representing the amount of images within the provided spritesheet over the y axis.
*** startFrame (optional):** Numeric value representing at which frame index of the spritesheet the animation should start. Default = -1 and indicates the first frame.
*** imageFitting (optional):** The ImageFitting enum value defines how the image is being applied to the particle div element and should usually not be altered. Default = ImageFitting.CONTAIN.

8. ParticleColorfilterData (experimental) 
The ParticleColorfilterData object allows for altering or adding color to the look of the particle as well as modifying hue, saturation and brightness. This is an experimental component. Be aware that some cases might yield odd results.

*** hueRotate (optional):** Numeric value representing the amount (in degrees) of hue shifting that must occur to the particle's appearance. Default = 0 and indicates no hue-shift.
*** color (optional):** A Color object being overlayed ontop of the particle functioning as a colored filter. Due to difficult color calculations on greyscale images/colors, there might be a slight offset in hue, saturation and brightness. However, this offset can be accounted for by providing an offsetted color, or tweaking the hue, saturation and brightness values. You will notice that by only applying color, you wont have much control over saturation and brightness that comes from the color object. Default = null and indicates no color.
*** noise (optional):** If a color was provided, random offset/noise will be applied to the color and values can range [0, 360]. If no color was provided, but hueRotate was, this noise will be applied to the hueRotate field. Smaller values yield better results in this case. Default = -1 and indicates no noise.
*** contrast (optional):** Numeric value that alters the contrast of the appearance of the particles. Values between [0,100] are valid. Default = -1 and indicates no contrast alterations.
*** saturation (optional):** Numeric value that alters the saturation of the appearance of the particles. Beware that this value takes effect on the provided base color/image so lower values [0,2] are most meaningful. However, values between [0,100] are valid. Default = -1 and indicates no altered saturation.
*** brightness (optional):** Numeric value that alters the brightness of the appearance of the particles. Values between [0,1000] are valid. Default = -1 and indicates no altered brightness.

---

7. Behavior objects

Behavior object are objects that describe the continuous behavior of a certain type that a particle expresses. Each behavior type acts on its corresponding data object. For example,

ParticleFlipbookBehavior acts on ParticleFlipbookData and ParticleDirectionalBehavior acts on ParticleDirectionData.

1. ParticleDirectionalBehavior 
The particledirectionalbehavior allows for particle movement in a certain direction. It acts on the ParticleDefaultData and has no constructor parameters as all data it requires is already stored in the data object. It does however still need to be created and applied to the emitter.

2. ParticleSizeByLifeBehavior 
This behavioral type allows for gradual changes in size over the lifetime of the particles and acts upon ParticleDefaultData.

*** sizeMultipliersX:** An array of numbers representing the scalars through which the particle is going interpolate over its lifetime/provided duration and multiply as a scalar to its width defined in the ParticleDefaultData.
*** sizeMultipliersY:** An array of numbers representing the scalars through which the particle is going interpolate over its lifetime/provided duration and multiply as a scalar to its height defined in the ParticleDefaultData.
*** duration (optional):** Numeric value indicating the Time (in ms) it should take for the particle to run through the sizeMuliplier lists once. Default = -1, meaning that it runs through those size lists once during its lifetime.
*** scalarNoise (optional):** Numeric value which adds noise/random offset to the sizeMuliplier lists. Smaller values yield better results. Default = -1 and indicates no noise.
*** uniformNoise (optional):** Boolean value defining whether the scalarNoise, if provided, should scale both lists uniformly. Beware that this parameter can yield odd results for sizeMuliplier lists of unequal length.
*** sizeIterationCount (optional):** Numeric value which allows for a forced halt in running through the sizeMuliplier lists after having passed x numbers in those lists. It then remains the size where it halted. Default = -1 and means that no halting should occur.

3. ParticleGravityBehavior 
The ParticleGravityBehavior allows for adding gravity and acts upon ParticleDefaultData.

*** fieldStrength:** Numeric value representing the force by which the particles are being pulled downwards. Negative values allow for inverted gravity.
*** fieldStrengthNoise (optional):** Numeric value which adds noise/random offset to the fieldStrength parameter. Smaller values yield better results. Default = -1 and indicates no noise.

4. ParticleRotationBehavior 
Defines how the particles should be rotating or 'spinning'. This behavior object acts on the ParticleRotationData.

*** rotation:** Numeric value representing the speed by which the particle should rotate. Positive values yield counter clockwise rotation.
*** rotationNoise (optional):** Numeric value which adds noise/random offset to the rotation parameter. Lower values yield better results. Default = -1 and means no noise is being added.
*** allowReverse (optional):** Boolean value defining whether the rotationNoise is allowed to invert the rotation direction. Default = true.

5. ParticleOpacityByLifeBehavior 
The ParticleOpacityByLifeBehavior allows for alteration in the particle's opacity over time. It acts upon the ParticleOpacityData object.

*** opacityMultipliers:** An array of numbers between [0,1] representing the scalars through which the particle is going interpolate and multiply with its opacity value defined in the ParticleOpacityData.
*** opacityNoise (optional):** Numeric value whichn adds noise/random offset to the

opacityMultipliers. Smaller values yield better results. Default = -1 and indicates no randomness.
*** duration (optional):** Numeric value representing the time (in ms) it should take for the particle to run through the opacityMultipliers once. Default = -1 and indicates that it runs through the opacity list once during its lifetime.
*** opacityIterationCount (optional):** Numeric value which allows for a forced halt in running through the opacityMultipliers after having passed x numbers in this opacity list. It then remains the opacity value where it halted. Default = -1 and indicates no halting.

6. ParticleWindBehavior 
The ParticleWindBehavior object allows to mimic the effects of wind. It allows just like directionalbehavior for movement to the particle, but the wind behavior allows for wavey movement. It acts upon the ParticleDefaultData.

*** angles:** A numeric array of angles (in degrees) representing the angles towards the wind blows and through which the particles are going interpolate and be blown towards.
*** cycleDuration (optional):** Numeric value representing the time (in ms) it should take for the particle to run through the angles list once. Default = -1 and means that it runs through it once during its lifetime.
*** windSpeed (optional):** Numeric value representing the power/windspeed by which the wind will blow towards the direction the particle is currently at in its angles list. Default = 1.
*** deepRandom (optional):** Boolean value that if true, randomizes all angles by an angle between the biggest and smallest angle provided in the angles list. Default = false.
*** shallowRandom (optional):** Boolean value that if true, randomizes each angle by a new random angle between its provided neighbouring angles. Default = false.
*** considerDefault (optional):** Boolean value that if true, the initial direction as defined in the ParticleDirectionData will not be discarded. Default = false.
*** overrideInitialDirection (optional):** Boolean value that if true, the initial direction as defined in the ParticleDirectionData is overridden by the first angle in the angle list. If false, the initial direction will be appended to the angles list. Note that this parameter is only valid when the considerDefault parameter is true. Default = false.

7. ParticleColorShiftBehavior (Experimental) 
The ParticleColorShiftBehavior allows for a gradual change in what the ColorFilterData only initializes once, when it comes to hue, saturation, brightness and contrast. This behavior object acts on the ColorFilterData. This is an experimental component and some configurations might yield odd results. This component can be used alongside the ParticleColorfilterBehavior in order to tweak results.

*** hues (optional):** A numeric array of hue-values between [0,360] through which the particles are going interpolate and an updates value is going to be assigned to the ColorFilterData. Beware that These updates hues are going to be added/subtracted from what was already configured in the ColorFilterData.
*** contrasts (optional):** A numeric array of contrast-values between [0,100] through which the particles are going interpolate and take on this interpolated contrast value for each moment in time over its life. Smaller values [0,2] yield better results.
*** saturations (optional):** A numeric array of saturation-values between [0,100] through which the particles are going interpolate and take on this interpolated saturation value for each moment in time over its life. Smaller values [0,2] yield better results.
*** brightnesses (optional):** A numeric array of brightness-values between [0,1000] through which the particles are going interpolate and take on this interpolated brightness value for each moment in time over its life. Smaller values [0,2] yield better results.
*** forceForwardHue (optional):** Given that hue is expressed as an angle, this boolean value will make sure a hue shift from 290 to 30 will not count downwards, but internally upwards from 290 to 360+30. Default = true and indicates an upwards shift only.
*** duration (optional):** A numeric value describing the duration over which these transformations are supposed to occur. Default = -1 and indicates that the transformations will occur over the lifetime of the particle.

8. ParticleColorfilterBehavior (Experimental) 
The ParticleColorfilterBehavior allows for a gradual change in what the ColorFilterData only

initializes once, when it comes to color. This behavior object acts on the ColorFilterData. This is an experimental component, and when it comes to providing colors, there might be some incorrect offset occuring due to difficult css color calculations upon already colored surfaces. These values can be tweaked using the ParticleColorShiftBehavior.

*** colors:** An array of color-objects (See colors further) through which the particles are going interpolate and take on this interpolated color overlay for each moment in time over its life.
*** duration (optional):** Numeric value representing the time (in ms) it should take for the particle to run through the colors once. Default = -1 and means that it runs through those colors once during its lifetime.
*** randomStartColor (optional):** Boolean value defining whether a random color from the colors list is being taken to start from. Default = false and indicates no randomisation.
*** startIndex (optional):** Similar to randomStartColor, the color at this index will be taken as start color. Default = -1 and means the first color.
*** colorIterationCount (optional):** Allows for a forced halt in running through the colors after having passed n colors in this list. It then remains the color value where it halted. Default = -1 and indicates no halting.

9. ParticleFlipbookBehavior 
The ParticleFlipbookBehavior object allows flipping through the provided spritesheet in the provided ParticleFlipbookData object in order to display an animation.

*** speed:** Numeric value representing the time (in ms) between each frame jump.
*** speedNoise (optional):** Numeric value which adds randomness/noise to the speed parameter. Lower values yield better results. Default = -1 and indicates no randomness.
*** endingFrameCount (optional):** Numeric value which allows for a forced halt in running through the spritesheet frames after having passed n frames in the image. It then remains stuck on the frame where it halted. Default = -1 and indicates no halting.

19. ParticleRotationByDirectionBehavior 
This behavior object allows the rotation to always be set to match the direction that the particle is going towards. The object acts on the rotation data object and does not function in combination with the ParticleRotationBehavior.

*** offset (optional):** Numeric value which adds an additional offset (in degrees) to the rotation which is being matched to the particle's direction. Default = -1 and indicates no offset.

---

8. Color, Pivot, PositionUnit, ImageFitting

There are several constants that are worth adressing. These are Color, Pivot, PositionUnit and ImageFitting. These are enums that hold constant values and are sometimes required during configuration of data objects and or behavioral objects.

1. Color
Colors are fairly self explanatory. It is however important to note that when a color is requested, i must be a PollenFX Color object. There are 2 methods of calling its constructor.

*** hex string value:** new Color('#45F18B')
*** RGB values:** new Color(125,251,66)

2. Pivot
When a Pivot enum is required, this will allow the user to configure the pivotpoint of the particle. For example, a pivot placed in the bottom-right corner will scale the particle outwards to the upper left corner when scaling.

*** Pivot.BEGIN:** Means either top, or very left border of the particle container.

*** Pivot.CENTER:** Means either vertical, or horizontal center of the particle container.
*** Pivot.END:** Means either bottom, or very right end of the particle container.

3. PositionUnit
When working with positions or dimensions, we sometimes want this to be expressed as a proportion of its parent, the document or in absolute pixels.

*** PositionUnit.PERCENTAGE:** Refers to an expression as a percentage of its parent. The css equivalent of this would be '%'.
*** PositionUnit.PIXEL:** Refers to an absolute value in pixels. The css equivalent of this would be 'px'.
*** PositionUnit.VIEW_WIDTH:** Refers to an expression as a percentage of the screen width. The css equivalent of this would be 'vw'.
*** PositionUnit.VIEW_HEIGHT:** Refers to an expression as a percentage of the screen height. The css equivalent of this would be 'vh'.
*** PositionUnit.VIEW:** Both VIEW_WIDTH and VIEW_HEIGHT combined when there is no distinction allowed between the two.

4. ImageFitting
When working with images, it can be useful to specify how the image has to be applied to the particle. These directly entail a css stylerule.

*** ImageFitting.COVER:** Represents the background-size : "cover" css property.
*** ImageFitting.FIT:** Represents the background-size : "100% 100%" css property.
*** ImageFitting.CONTAIN:** Represents the background-size : "contain" css property.

---

9. Frontend framework compatibility

1. Angular
Given that this library is written in plain javascript, one might need to make a slight adjustment in order to use it in a framework such as angular
1. Add the script to your project.
2. Add a reference to the script in your angular.json file as shown below:

```
"projects": {
  "Webframework": {
    "projectType": "application",
    "schematics": {
      "@schematics/angular:component": {
        "style": "scss"
      }
    },
    "root": "",
    "sourceRoot": "src",
    "prefix": "framework",
    "architect": {
      "build": {
        "builder": "@angular-devkit/build-angular:application",
        "options": {
          "outputPath": "dist/webframework",
          "index": "src/index.html",
          "browser": "src/main.ts",
          "polyfills": [
            "zone.js"
          ],
          "tsConfig": "tsconfig.app.json",
          "inlineStyleLanguage": "scss",
          "assets": [
            "src/favicon.ico",
            "src/assets"
          ],
          "styles": [
            "src/styles.scss"
          ],
          "scripts": [
            "src/assets/scripts/pollenFX.js"
          ]
        },
```

3. In the component in which you wish to use PollenFX, add a reference to PFX, declaring its existance.

```
1  import { Component, OnInit } from '@angular/core';
2  import { FormBuilder, FormGroup, Validators } from '@angular/forms';
3  import { AuthenticationService } from '../../../services/authenticationService.service';
4  import { HttpErrorResponse, HttpResponse } from '@angular/common/http';
5  import { RoutingService } from '../../../services/routingService.service';
6  import { CookieService } from '../../../services/cookieService.service';
7
8  declare const PFX: any;
9
10 @Component({
11   selector: 'app-login',
12   standalone: false,
13   templateUrl: './login.component.html',
14   styleUrl: './login.component.scss'
15 })
16
17 export class LoginComponent implements OnInit {
18
19   /* PARAMETERS */
20   private _loginForm!: FormGroup;
```

4. From now on, the engine can be used, albeit with every object taken from the PFX

constant.

```
private configureLeaves(): void {
  // anchor
  const anchor : HTMLElement = document.querySelector(".bd-landscape");

  // setup
  this.fxManager = new PFX.FXManager(false);
  let origin = new PFX.CircularEmitterOrigin((anchor.offsetWidth / 2), (anchor.offsetHeight /
  const emitter = new PFX.EmitterShoot(origin, 100, -1, 15000, -1, -1, 500)

  // data
  const defaultData = new PFX.ParticleDefaultData(10, 10, origin, 2, 2, true);
  const customCssData = new PFX.ParticleCustomCssData("", 100, 500);
  const directionData = new PFX.ParticleDirectionData(-90, 4, 360, 2);
  const rotationData = new PFX.ParticleRotationData(0, 360);
  const imageData = new PFX.ParticleImageData("./../../assets/images/backdrop/backdrop_leaf.pr
  emitter.addParticleData(defaultData);
  emitter.addParticleData(directionData);
  emitter.addParticleData(customCssData);
  emitter.addParticleData(rotationData);
  emitter.addParticleData(imageData);

  // behavior
  const sizebylifeBehavior = new PFX.ParticleSizeByLifeBehavior([0.5,1,1.5,0],[0.5,1,1.5,0]);
  const rotationBehavior = new PFX.ParticleRotationBehavior(1, 1, true);
  const directionBehavior = new PFX.ParticleDirectionalBehavior();
  const opacityByLifeBehavior = new PFX.ParticleOpacityByLifeBehavior([0,1,1,1,0]);
  const particleWindBehavior = new PFX.ParticleWindBehavior([90,180,270,-90, 90], -1, 10);
  emitter.addParticleBehavior(directionBehavior);
  emitter.addParticleBehavior(rotationBehavior);
  emitter.addParticleBehavior(sizebylifeBehavior);
  emitter.addParticleBehavior(opacityByLifeBehavior);
  emitter.addParticleBehavior(particleWindBehavior);
```

---

10. Tips & tricks

1. Anchor choice
When choosing an element to function as an anchor, it is recommended to make a dedicated div element. Obscure, non-div elements might yield unwanted results.

2. Anchor styling
It is recommended to not pick an anchorelement with lots of exotic styling applied to it or that is positioned relative/absolute. Doing this could yield unwanted results.

3. Cascading css
Ultimately, particle effects are just transformed divs, so beware of any cascading css. These might ruin the particle configurations due to cascading styling.

4. PollenFX script inclusion
When linking to the pollenFX script in order to use it, make sure it is inserted above the script in which you are going to use it.

---

11. Additional functionality

1. FXItemID
When adding an emitter to the FXManager, one can add an ID. Later, perhaps in order to add extra custom functionality, one can request this emitter by its ID in order to modify it.

```
38      /* Add the emitter to the FXManager */
39      fxManager.addEmitter(emitter, "myFirstEmitter")
40
41      /* Retrieve the emitter by its ID */
42      fxManager.getFxItemById("myFirstEmitter");
```

2. EmitterBox debugging

In order to figure out where the emitterbox is located or what size it has, one can enable the debug option as shown in the image below. This will highlight the emitterBox with a green-ish color.

```
/* 1. Setup */
document.addEventListener('DOMContentLoaded', () => {
    fxManager = new FXManager(true);
    FXManager.devConfig.DEBUG = true;
    fxManager.addEmitter(createEmitter(), "my_emitter");
    this.start(0);
});
```

3. Retrieving FPS

The current running framerate can be obtained from the FXManager.

```
43  /* Core loop */
44  function start(nowTime) {
45      fxManager.act(nowTime);
46      console.log(fxManager.getFPS());
47      window.requestAnimationFrame(this.start);
48  }
49
```

---

12. An extensive code snippet

Finally, here is a more in-depth example of the creation of a particle effect. Beware that the examples below might make use of resources such as images that are not provided in the demos.

```
/**
 * Setup
 */
let fxManager;
document.addEventListener('DOMContentLoaded', function () {
    fxManager = new FXManager();
    fxManager.addEmitter(getEmitter(), "myEmitter");
    start(0);
});


/**
 * Function emitter creation function
 */
function getEmitter() {

    /* Anchor, origin and emitter */
    let anchor = document.getElementById("my_element");
    let origin = new CircularEmitterOrigin(50, 0, 100, 20, -1, -1, true, anchor);
    let emitter = new EmitterShoot(origin, 400, 2000, 700, 1000, 2);

    /* Data */
    const defaultData = new ParticleDefaultData(14, 7, origin, 1, 1, true);
    const cssData = new ParticleCustomCssData("background-color:red;border:1px solid black; border-radius:5px;",
    const directionData = new ParticleDirectionData(90, 7, 50, 4);
    emitter.addParticleData(cssData);
    emitter.addParticleData(defaultData);
    emitter.addParticleData(directionData);

    /* Behavior */
    const directionBehavior = new ParticleDirectionalBehavior()
    const gravityBehavior = new ParticleGravityBehavior(10, 4)
    const sizeByLifeBehavior = new ParticleSizeByLifeBehavior([0,1,0])
    const rotationByDirectionBehavior = new ParticleRotationByDirectionBehavior()
    emitter.addParticleBehavior(directionBehavior);
    emitter.addParticleBehavior(gravityBehavior);
    emitter.addParticleBehavior(sizeByLifeBehavior);
    emitter.addParticleBehavior(rotationByDirectionBehavior);

    return emitter;
}


/**
 * Core logic loop
 */
function start(runtimeInMs) {
    fxManager.act(runtimeInMs);
    window.requestAnimationFrame(this.start);
}

```